Operating Systems Technologies

(Tecnologias de Sistemas Operativos)

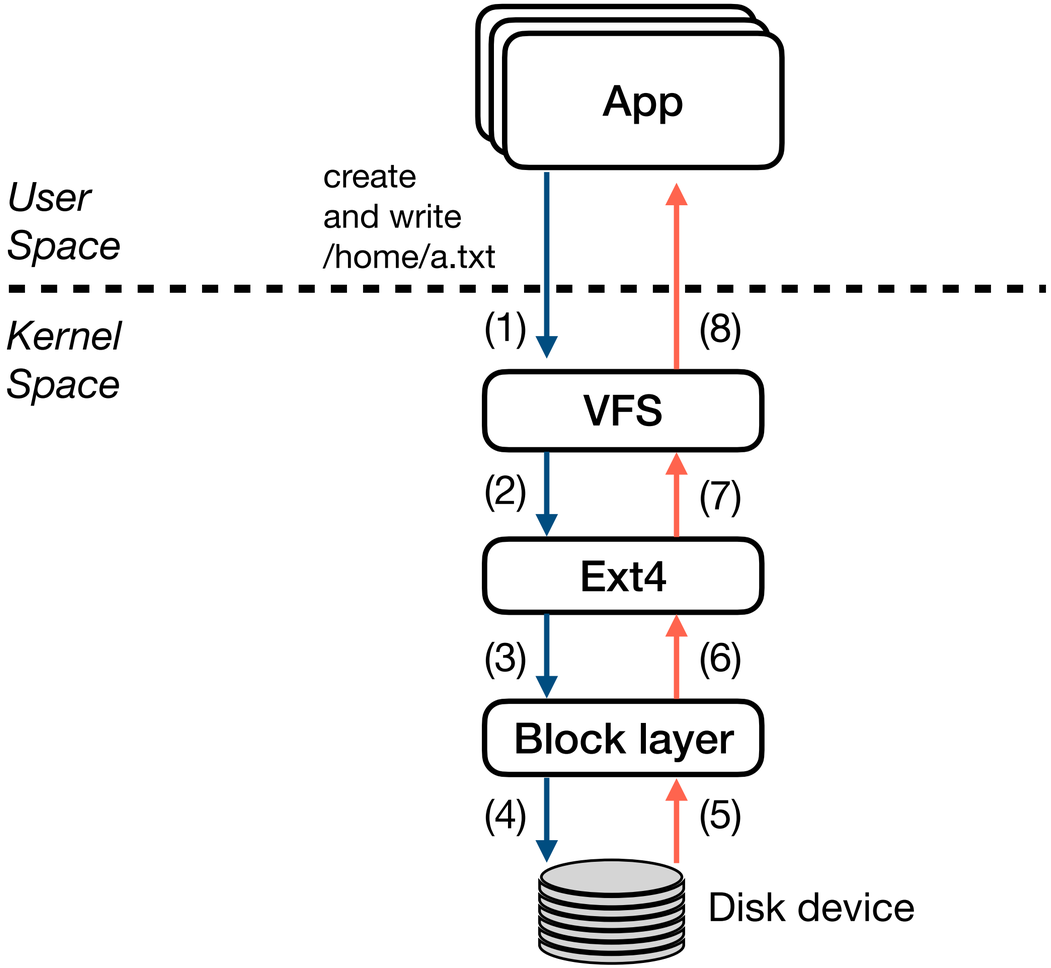
File System in User Space

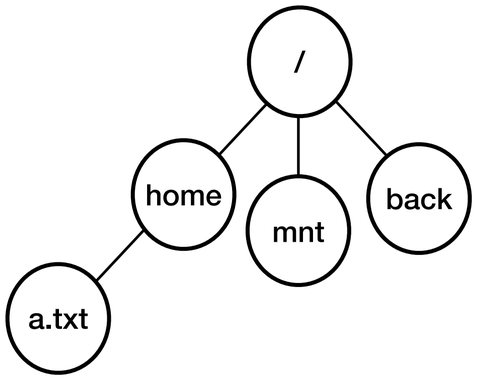
University of Minho

2025-2026

Kernel-based FS

Recap: flow of I/O requests





Subset of the tree directory of the Ext4 file system mounted over /

File Systems

Creating and extending file system behavior

- Implementing a new File System (FS) is a complex task
  - Kernel code is harder to write and debug than user space code!
- The same is true if one wants to extend an existing kernel-based implementation with new features, for instance
  - data encryption
  - data compression
  - remote storage
  - …
- Is there any alternative?

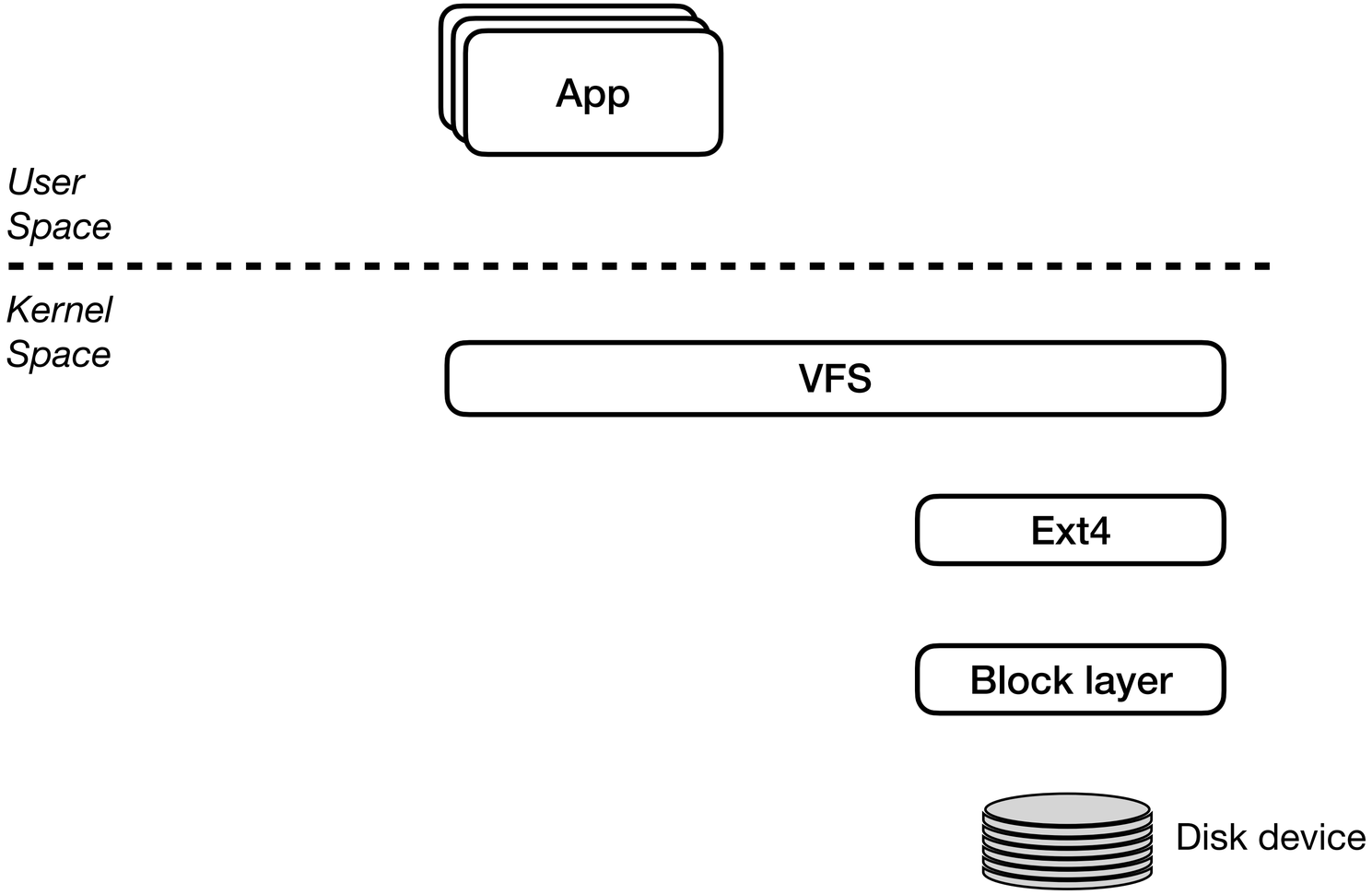
FUSE

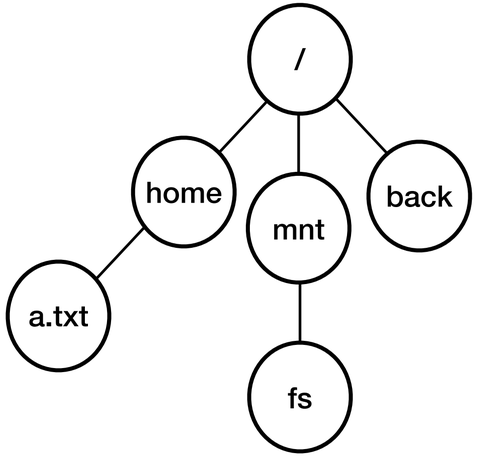
File System in Userspace

- The FUSE framework allows building custom FS implementations
  - It acts as an interface for user space programs to export a filesystem to the Linux kernel
- It consists of two components:
  - **FUSE kernel module:** implements the kernel code to provide a standard file system that handles requests from applications. Maintained in the regular kernel repositories
  - **libfuse user space library:** provides functions to mount/unmount the FS, read requests from the kernel, and send responses back. Available at https://github.com/libfuse/libfuse
- A FUSE FS can be implemented as a user space program that links with libfuse
  - What if we want to implement a FUSE FS that redirects I/O operations into another kernel-based FS (e.g., ext4) on the same computer (i.e., a **pass-through FS**)

FUSE Passthrough

Flow of I/O requests

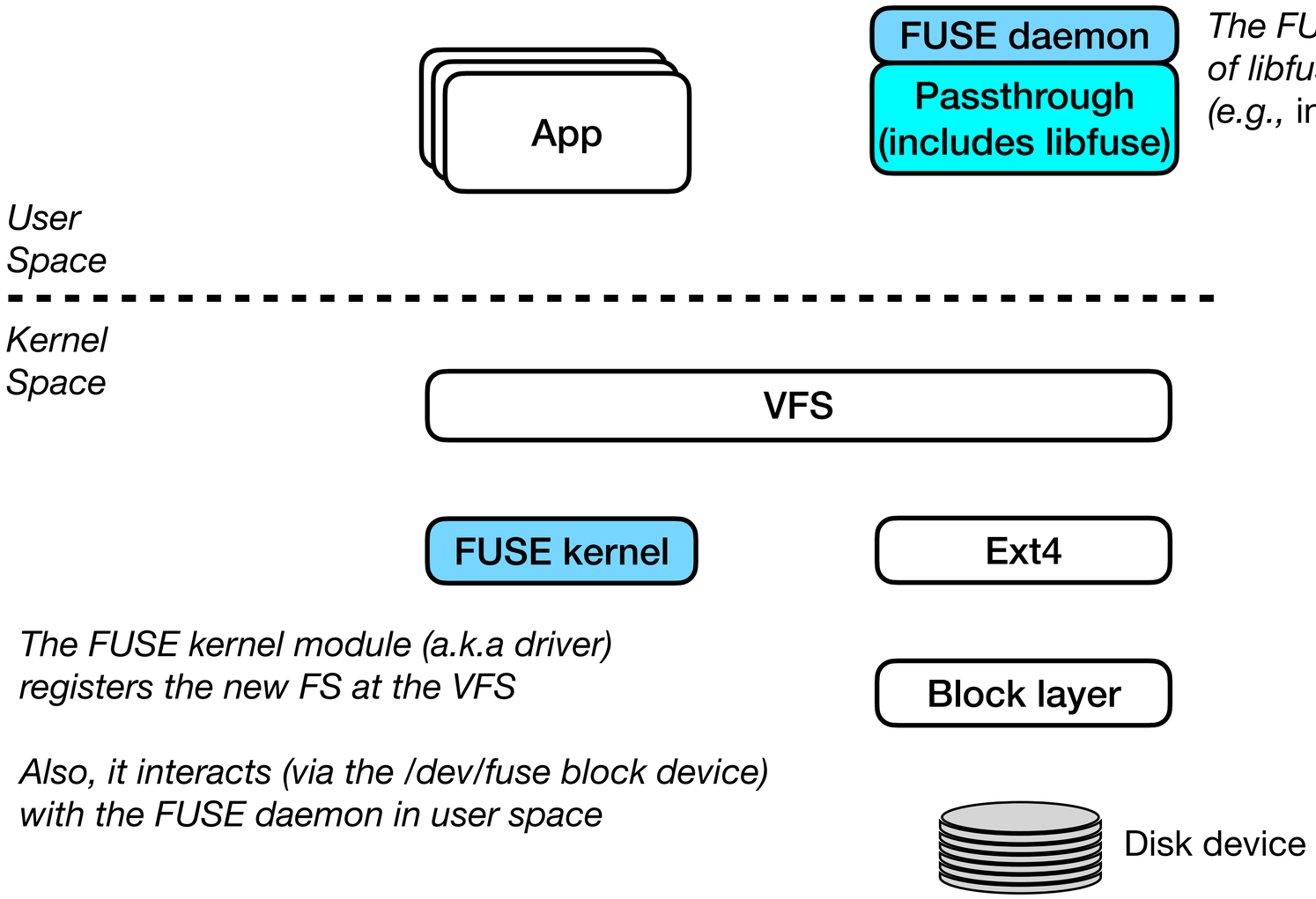




Ext4 file system mounted over /

FUSE Passthrough

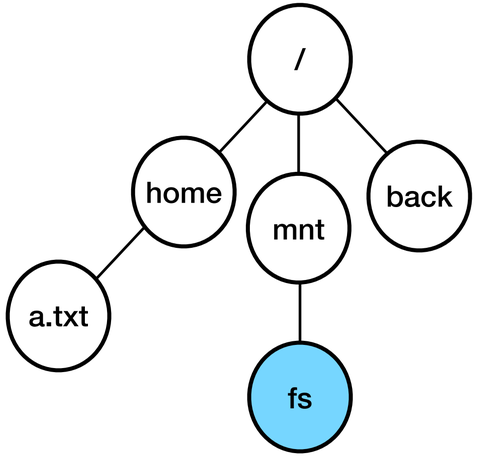
Flow of I/O requests



The FUSE daemon invokes specific functions of libfuse at the passthrough program (e.g., init the FS, file/directory operations*)*

The FUSE kernel module (a.k.a driver) registers the new FS at the VFS

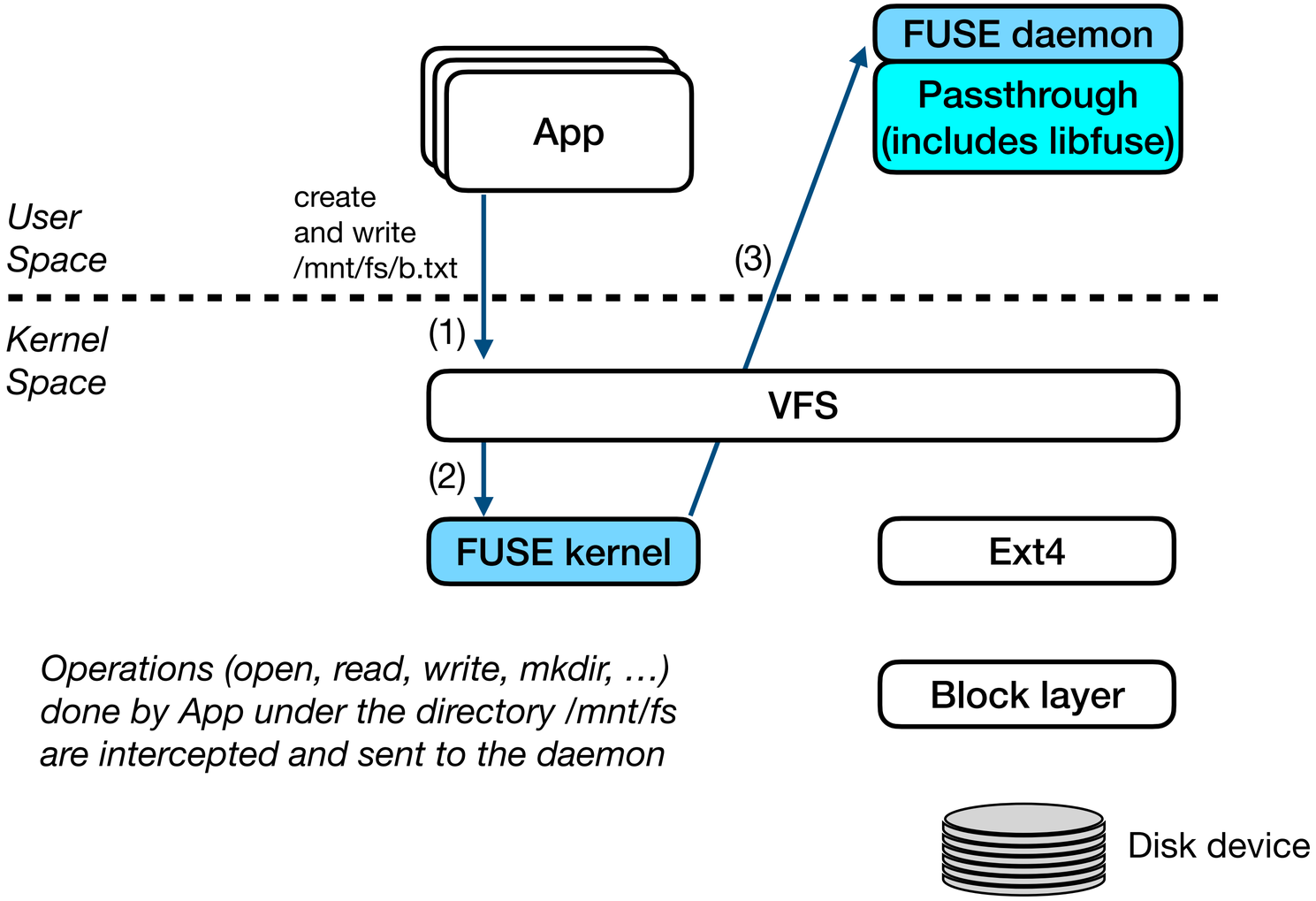
Also, it interacts (via the /dev/fuse block device) with the FUSE daemon in user space



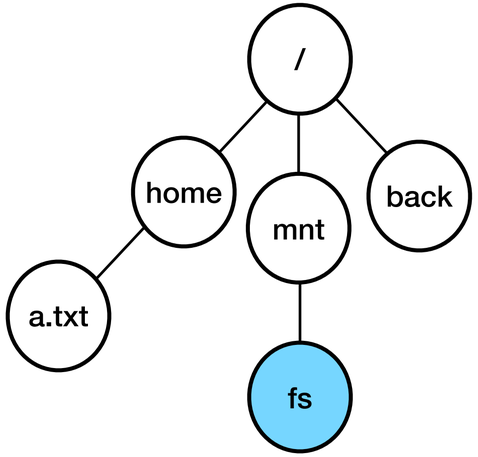
Ext4 file system mounted over /
FUSE passthrough mounted over /mnt/fs

FUSE Passthrough

Flow of I/O requests



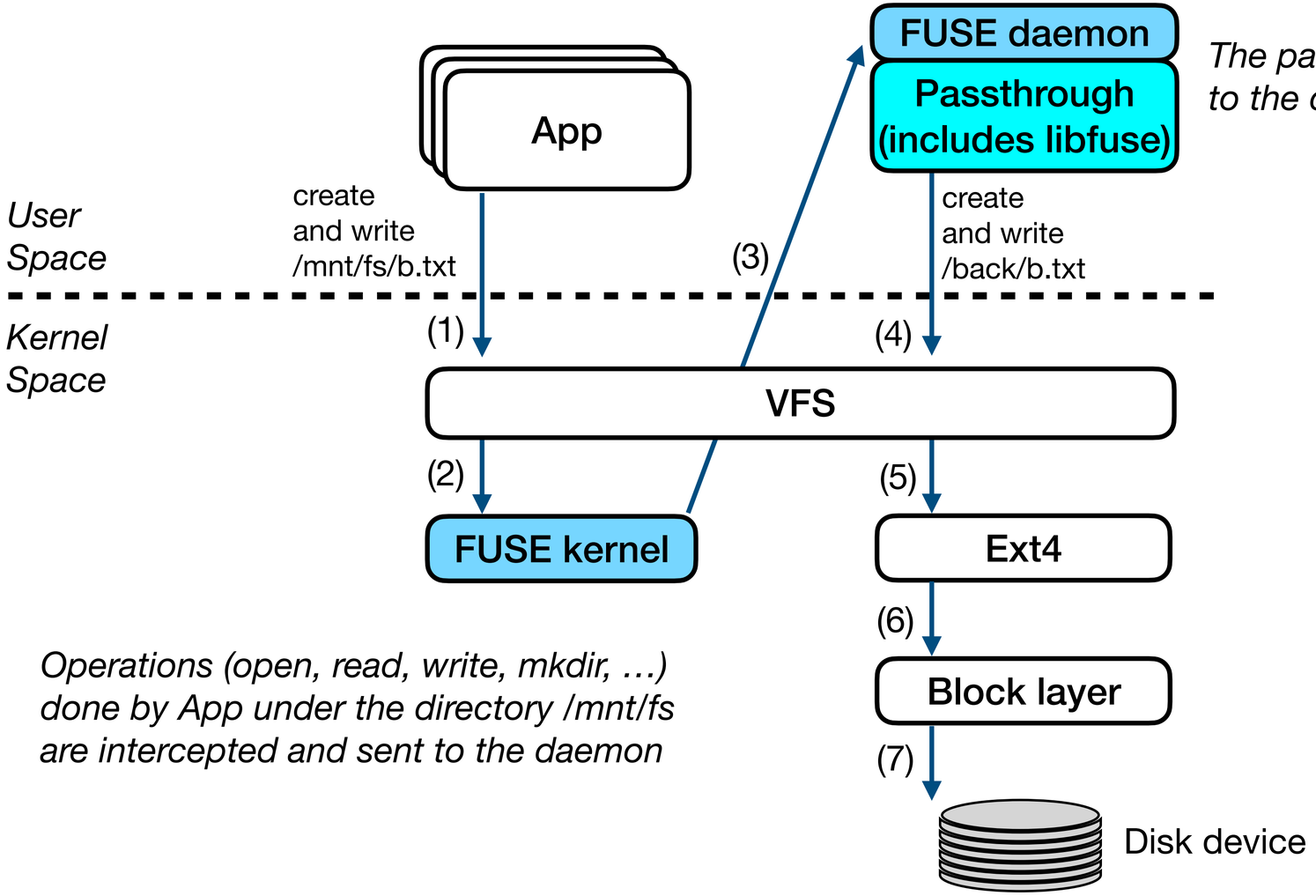
Operations (open, read, write, mkdir, …) done by App under the directory /mnt/fs are intercepted and sent to the daemon



Ext4 file system mounted over /
FUSE passthrough mounted over /mnt/fs

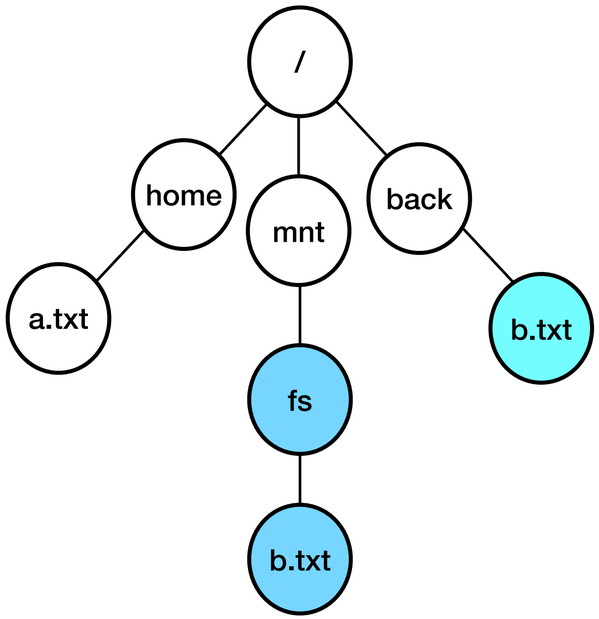
FUSE Passthrough

Flow of I/O requests



The passthrough FS redirects these operations to the directory /back under the control of ext4

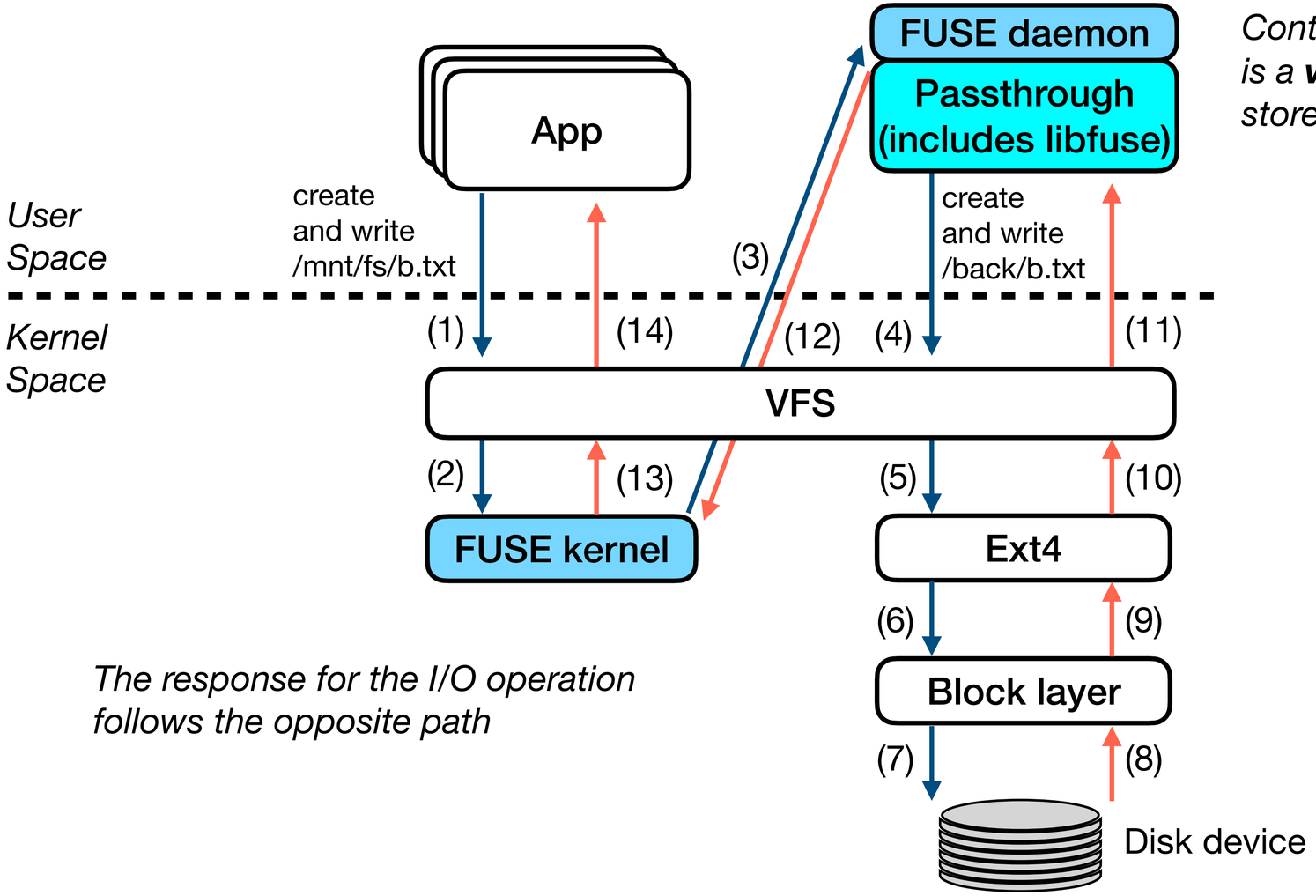
Operations (open, read, write, mkdir, …) done by App under the directory /mnt/fs are intercepted and sent to the daemon



Ext4 file system mounted over /
FUSE passthrough mounted over /mnt/fs

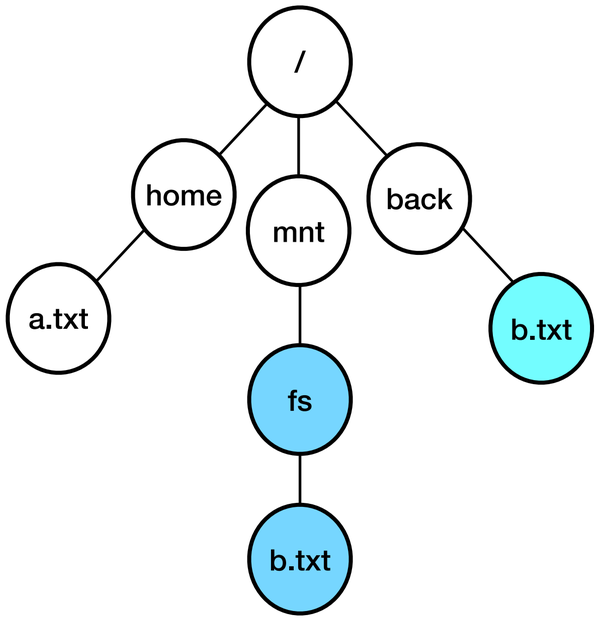
FUSE Passthrough

Flow of I/O requests



The response for the I/O operation follows the opposite path

*Content (i.e., files and directories) under /mnf/fs is a **virtual representation** of the actual content stored in /back/*



Ext4 file system mounted over /
FUSE passthrough mounted over /mnt/fs

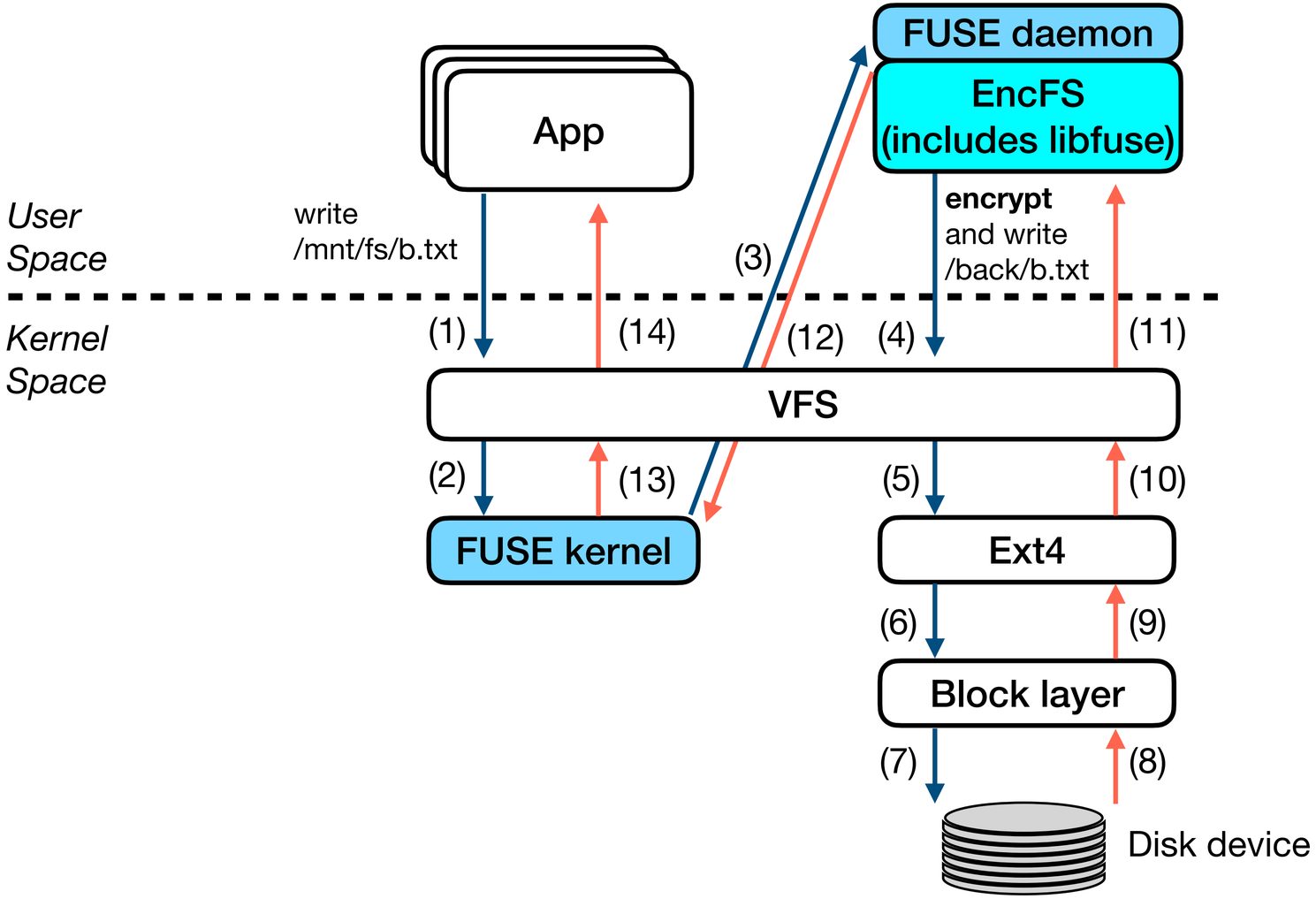
FUSE

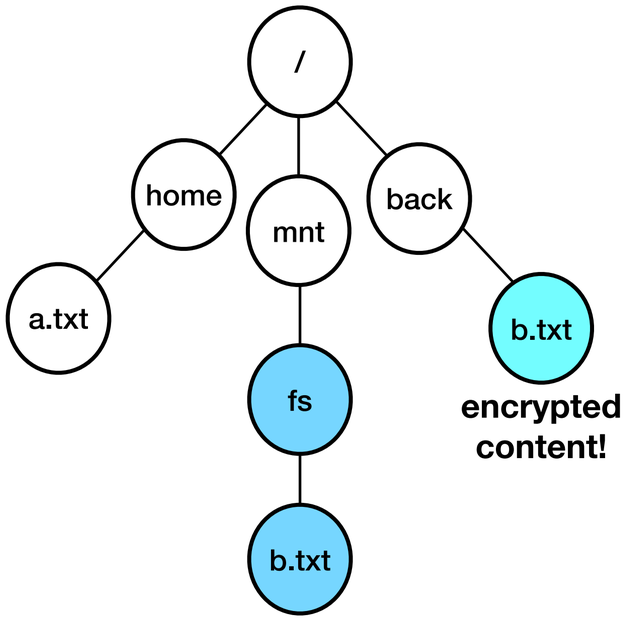
Use cases

- Nice trick! But where is FUSE useful other than writing an FS that writes to another FS?

Transparent Data Entryption

Some examples

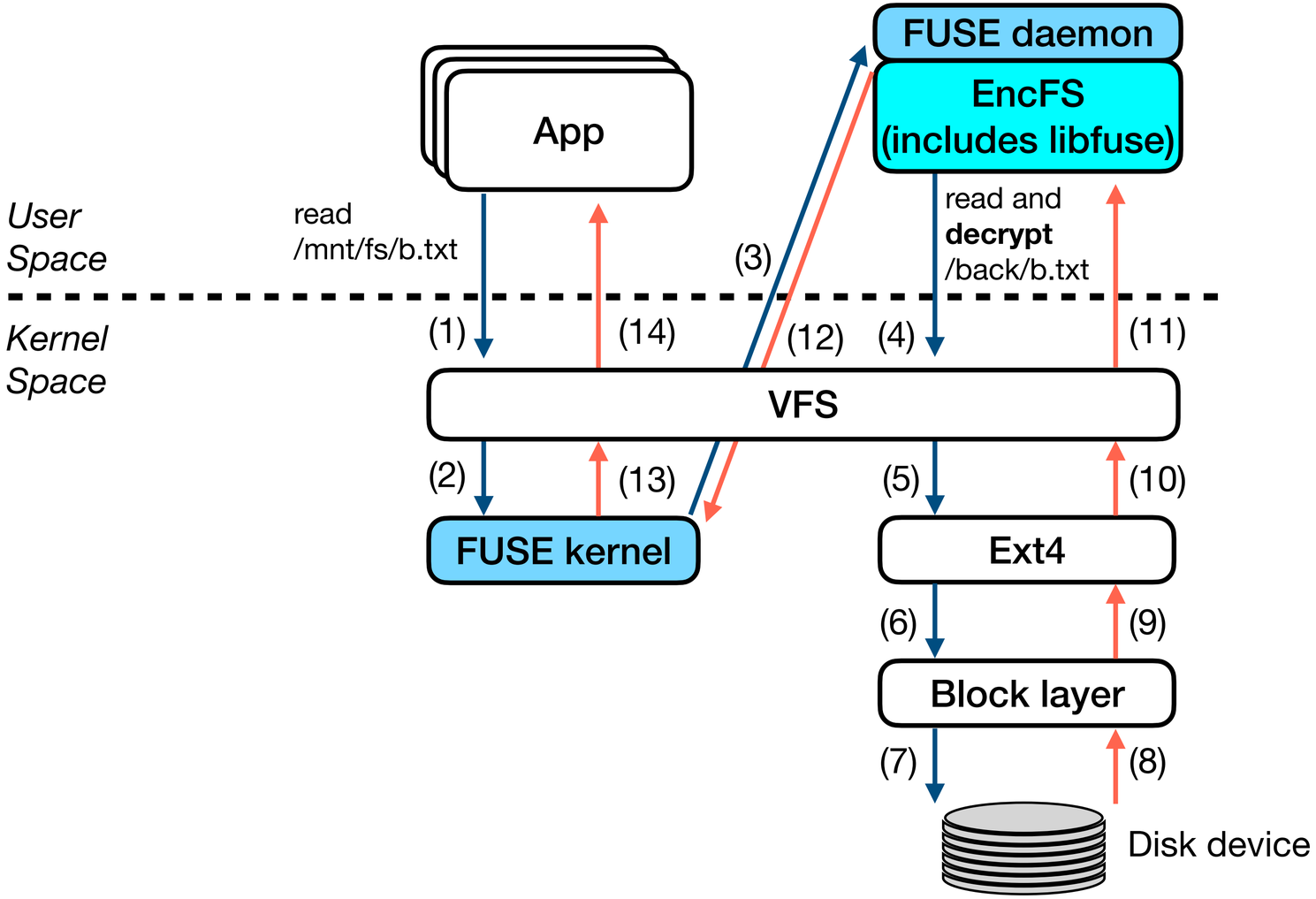


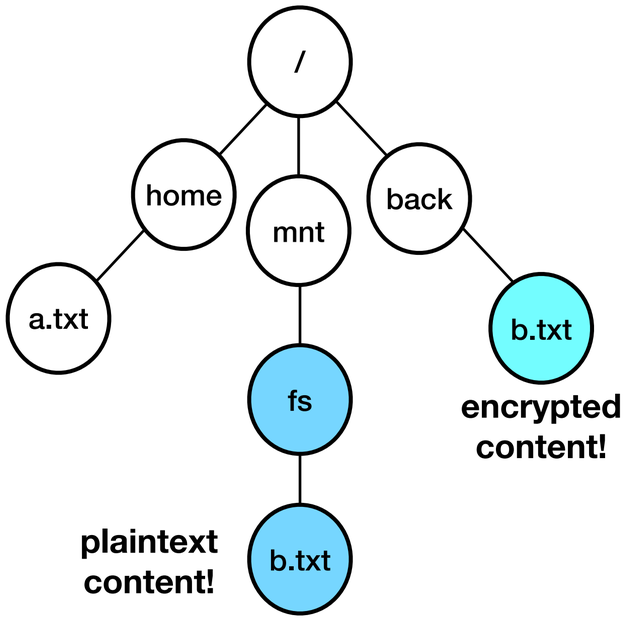


Ext4 file system mounted over /
EncFS mounted over /mnt fs

Transparent Data Entryption

Some examples

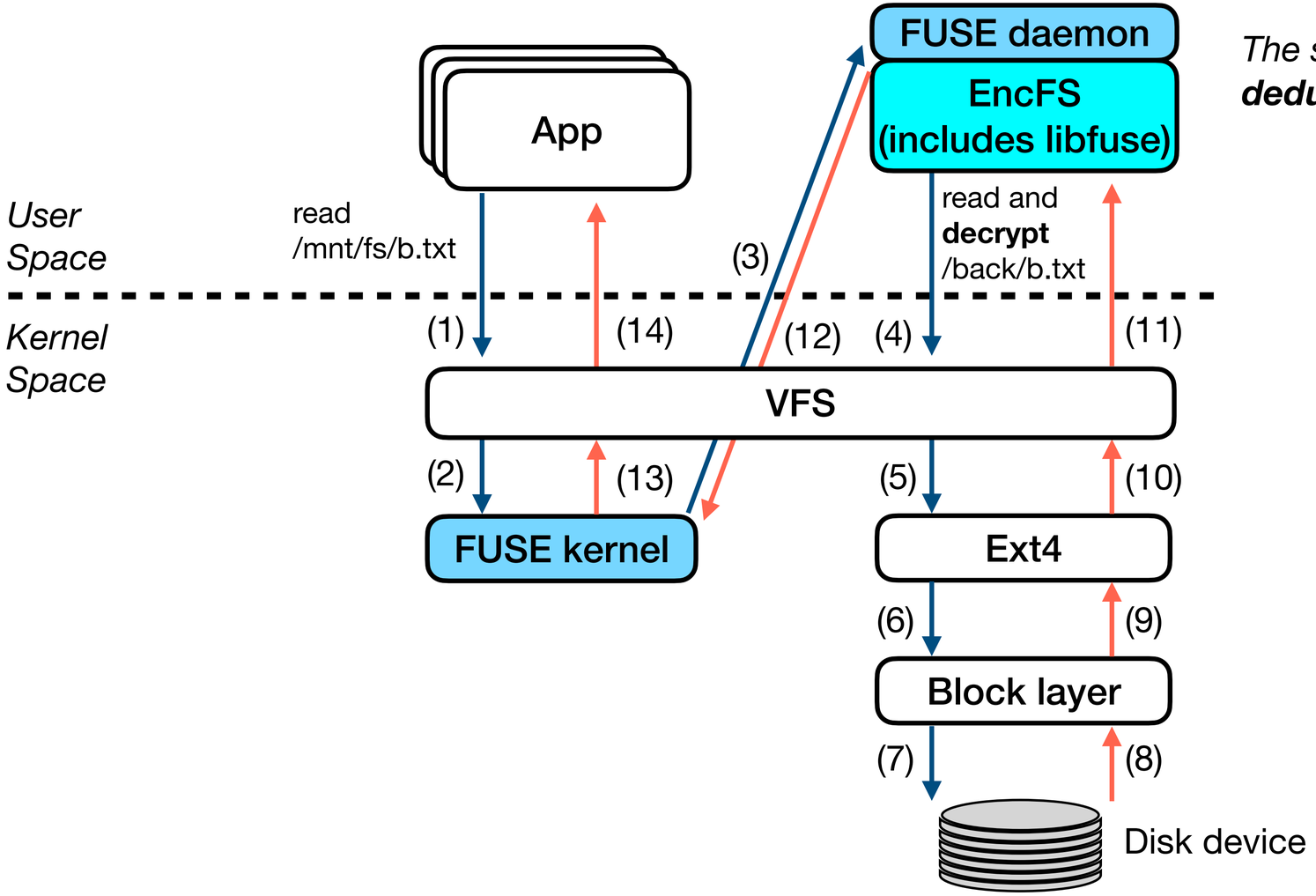




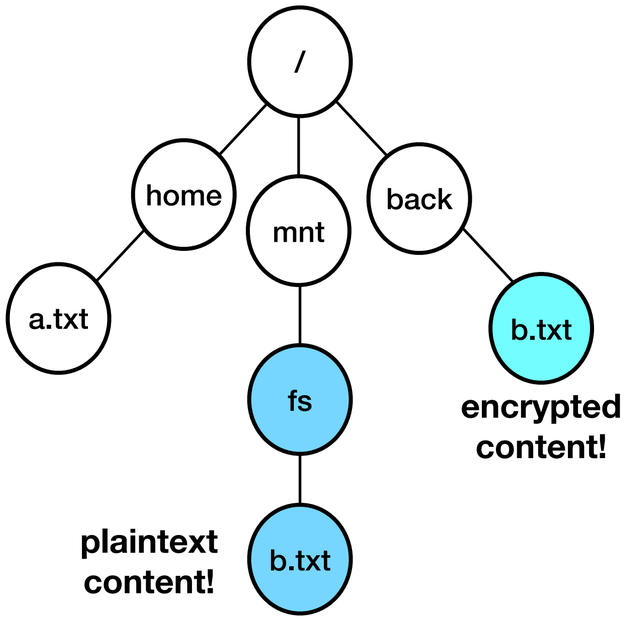
Ext4 file system mounted over /
EncFS mounted over /mnt fs

Transparent Data Entryption

Some examples



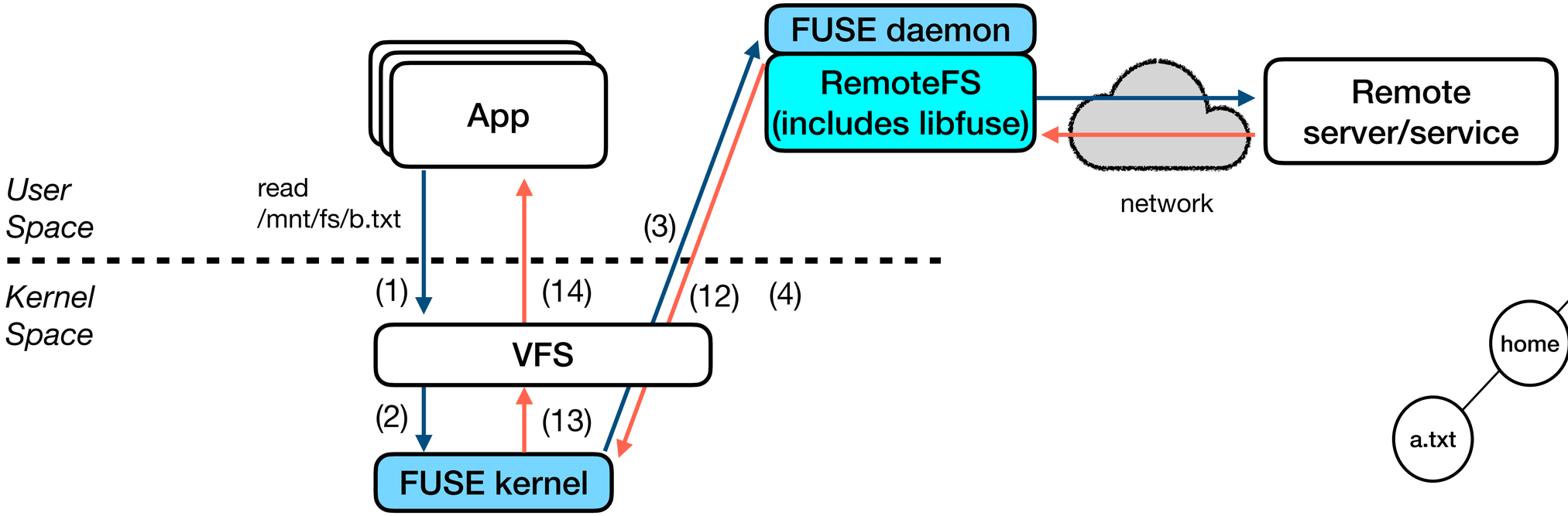
*The same trick can be applied to **data compression**, **deduplication**, and other **storage transformations**!*

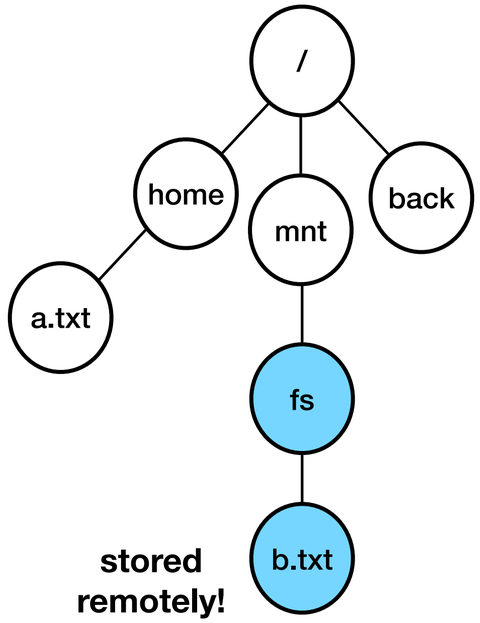


Ext4 file system mounted over /
EncFS mounted over /mnt fs

Remote File System

Some examples





RemoteFS mounted over /mnt fs

And one can even combine the previous examples,
e.g., ***compress*** *and* ***encrypt*** *data before* ***storing it remotely****!*

FUSE

APIs and structures

- libfuse provides two APIs to users
  - A **low-level asynchronous API** where programmers must work at the inode abstraction
  - A **high-level synchronous API**, built on top of the low-level one, where programmers work with file names and paths instead of inodes
- The ***struct fuse_operations*** contains a “table” of pointers to functions that define the behavior of the user space FS
  (init, destroy, read, write, mkdir, rename, …)

Considerations

Performance and correctness

- Performance overhead
  - You've probably noted that a FUSE-based FS requires extra communication (and switching) between user and kernel space
  - Along with extra (and costly memory copies)
- Correctness
  - When doing your FUSE-based implementation, be careful with what behavior is expected from a full-fledged FS
    - Is one keeping *rename()* atomic?
    - Does *fsync()* flush all cached data (and metadata) of the file?
    - ...
  - If some of these assumptions are not kept, be clear in your documentation!

More Information

- ***libfuse GitHub***. https://github.com/libfuse/libfuse
- ***libfuse API documentation***. https://libfuse.github.io/doxygen/index.html
- Vangoor et al. ***To FUSE or Not to FUSE: Performance of User-Space File Systems.*** FAST, 2017.
  - Section 2: Know more about FUSE's design and architecture
  - Sections 3, 4, and 5: Understand better FUSE's performance overhead and tuning

Questions?